Streaming AI/ML with Apache Kafka
Real-Time Patterns for Modern Intelligence

Paolo Patierno
Software Engineer @ IBM

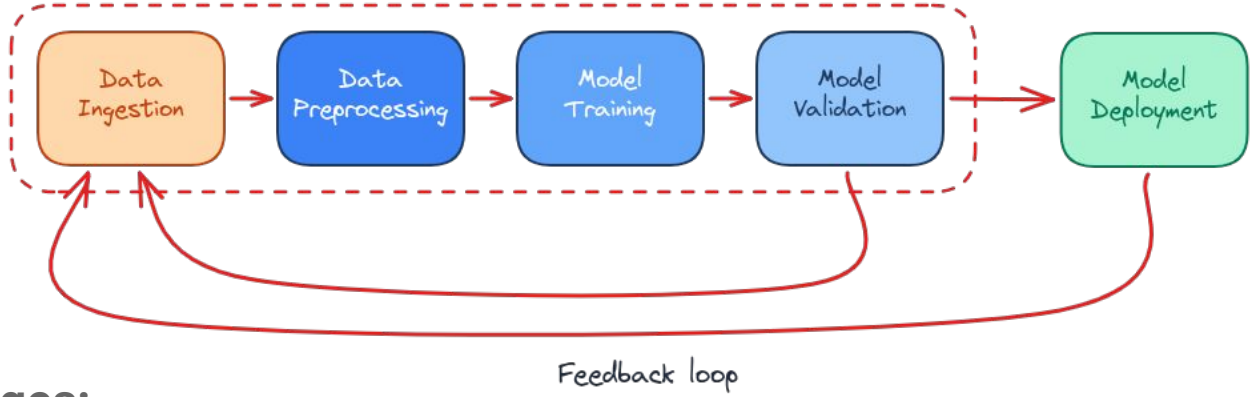
About me

- Software Engineer @ IBM
  - Working on Red Hat Streams for Apache Kafka
  - 100% open source
- Strimzi maintainer and co-founder
- Kafka contributor
- CNCF Ambassador
- Confluent Catalyst

The Challenge

AI and ML systems only create value through the data that feeds them

Traditional Machine Learning Pipeline

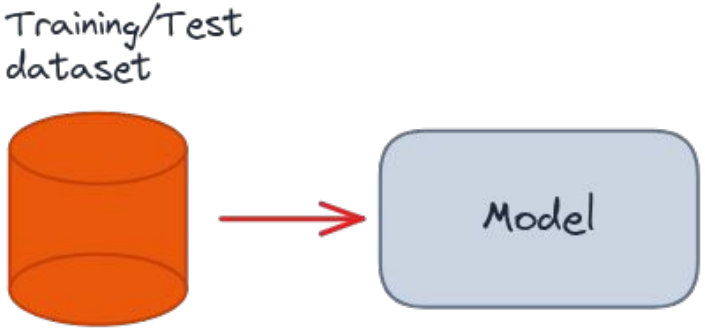


Four key stages:

1. Data Ingestion
2. Data Preprocessing
3. Model Training, Validation & Test
4. Model Deployment

Plus: Continuous feedback loop for monitoring and retraining

Batch Training: The Limitations



Problems with traditional batch approach:

- **Static datasets** are snapshots of the past
- **Batch processing** introduces latency
- **Model drift** degrades performance over time
  - Data distributions change, patterns shift
  - Predictions become less accurate

The Streaming Solution

From snapshots to continuous flows

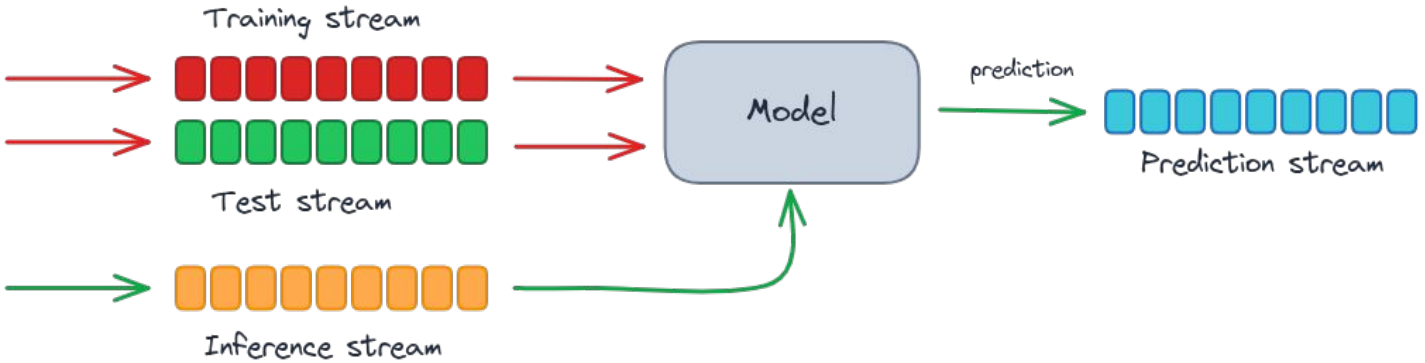
- Continuously updated training sets
- Dynamic validation in real-time
- Real-time feature extraction
- Immediate feedback loops
- Detect and respond to model drift

Apache Kafka enables this transformation

Part 1: ML Training & Validation Patterns

Streaming approaches for model development and continuous learning

Pattern 1: Training & Test Streams



Four Kafka topics structure:

- **Training Stream** - labeled data for model training
- **Test Stream** - labeled data for model validation and testing
- **Inference Stream** - unknown/unlabeled data for predictions
- **Prediction Stream** - model outputs

Benefits: Real-time preprocessing with Kafka Streams/Flink

Pattern 1: Message Examples (IoT Predictive Maintenance)

Training Stream (labeled)

```
Key: "sensor_A1234"
Headers:
  label: "failure"
Value:
{
  "temp": 85.5,
  "vibration": 12.3,
  "pressure": 102.7
}
```

Test Stream (labeled)

```
Key: "sensor_B5678"
Headers:
  label: "normal"
Value:
{
  "temp": 72.1,
  "vibration": 3.2,
  "pressure": 98.5
}
```

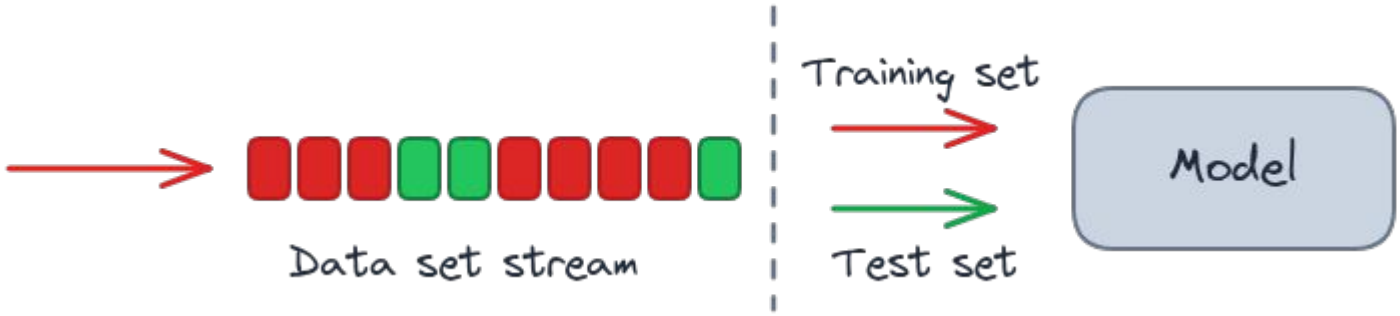
Inference Stream (unlabeled)

```
Key: "sensor_C9012"
Headers:
  request_id: "req_103045"
Value:
{
  "temp": 88.2,
  "vibration": 15.7,
  "pressure": 105.3
}
```

Prediction Stream (output)

```
Key: "sensor_C9012"
Headers:
  request_id: "req_103045"
  model_version: "v3.1.2"
Value:
{
  "prediction": "failure",
  "probability": 0.87,
  "ttf_hours": 48
}
```

Pattern 2: Dataset Splitting



Single stream → Multiple datasets

- **Time-based** - older events for training, newer for testing
- **Hash-based** - automatic random assignment, keeps entity data together
- **Entity-based** - pre-selected entity IDs, ensures entity integrity
- **Business-rule** - split by entity attributes (region, type, segment)

Pattern 2: Split Strategy Examples

Time-based Split

```
// Train on old data, test on new
cutoff = 1714521600000;
if (event.timestamp < cutoff) {
  send_to_training();
} else {
  send_to_test();
}

Example:
1708012345000 → Training
1715234567000 → Test
```

Hash-based Split (Automatic)

```
// Automatic random assignment
hash = hash(event.device_id) % 100;
if (hash < 80) {
  send_to_training(); // 80%
} else {
  send_to_test(); // 20%
}

Example:
device_A123 → hash=42 → Training
device_B456 → hash=87 → Test
```

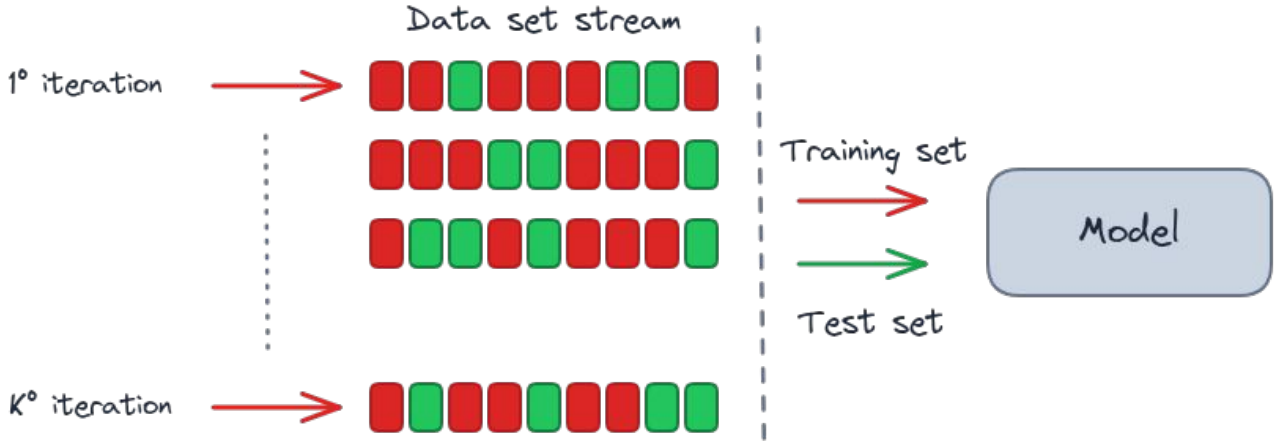
Entity-based Split (by ID)

```
// Pre-selected entity IDs
if (device_id in training_ids) {
  send_to_training();
} else if (device_id in test_ids) {
  send_to_test();
}

Example:
device_A (all data) → Training
device_B (all data) → Test
```

Business-rule Split (by Attribute)

```
// Split by entity attributes
if (event.location == "factory_1") {
  send_to_training();
} else if (event.location == "factory_2") {
  send_to_test();
}

Example:
All factory_1 devices → Training
All factory_2 devices → Test
```

Pattern 3: K-Fold Cross-Validation

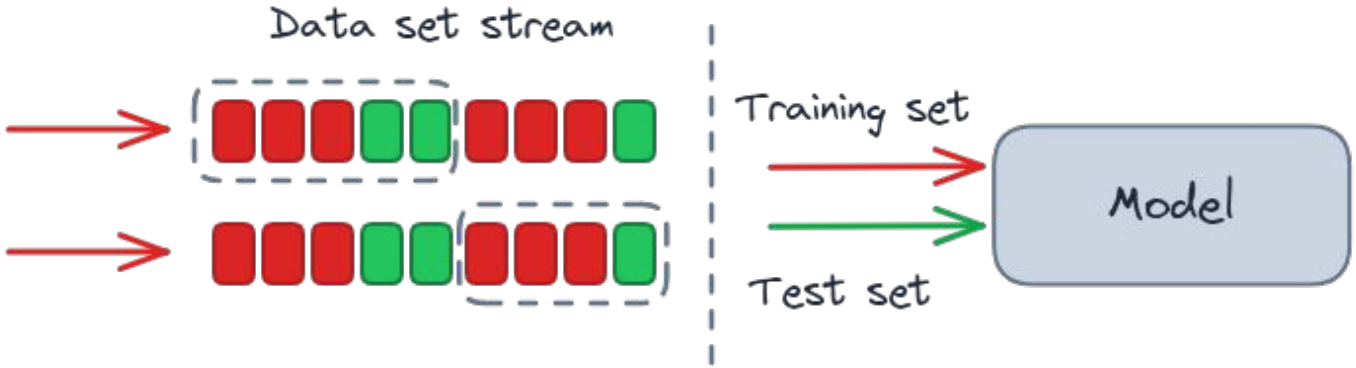


Stream rewind feature enables:

- Generate multiple folds for training/validation
- Repeatedly filter and split the stream
- Fine-tune (hyper)parameters across epochs and folds

Every data point used for both training and validation

Pattern 4: Windowing Techniques

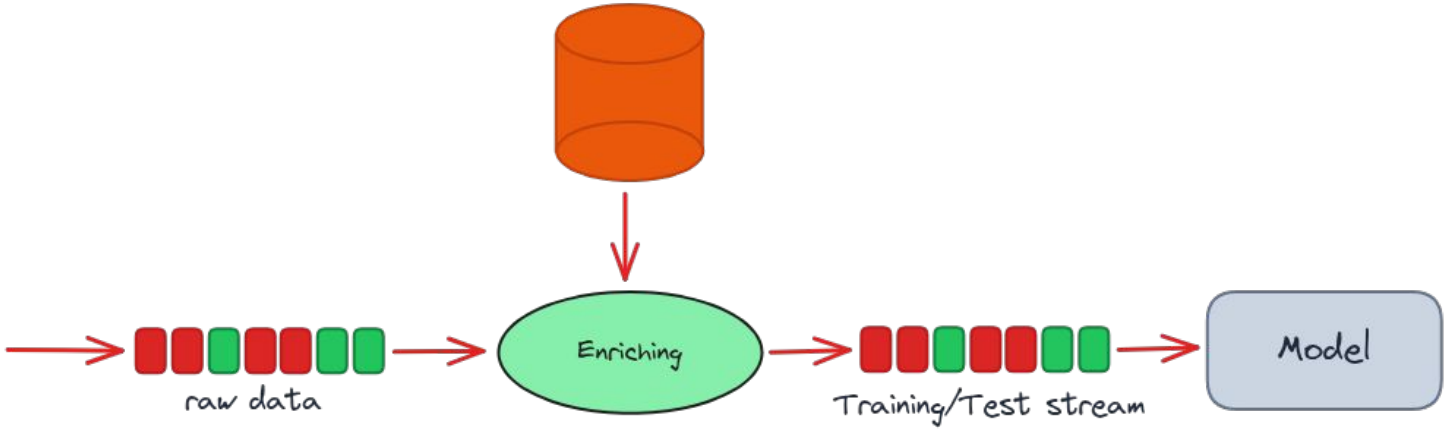


Boxing infinite streams:

- **Tumbling Windows** - fixed, non-overlapping time periods (e.g., 5-minute chunks)
- **Sliding Windows** - overlapping windows for continuous updates
- **Count-based Batching** - process after N records (e.g., every 1,000 records)

Makes infinite streams manageable for training and testing

Pattern 5: Data Preparation & Enrichment

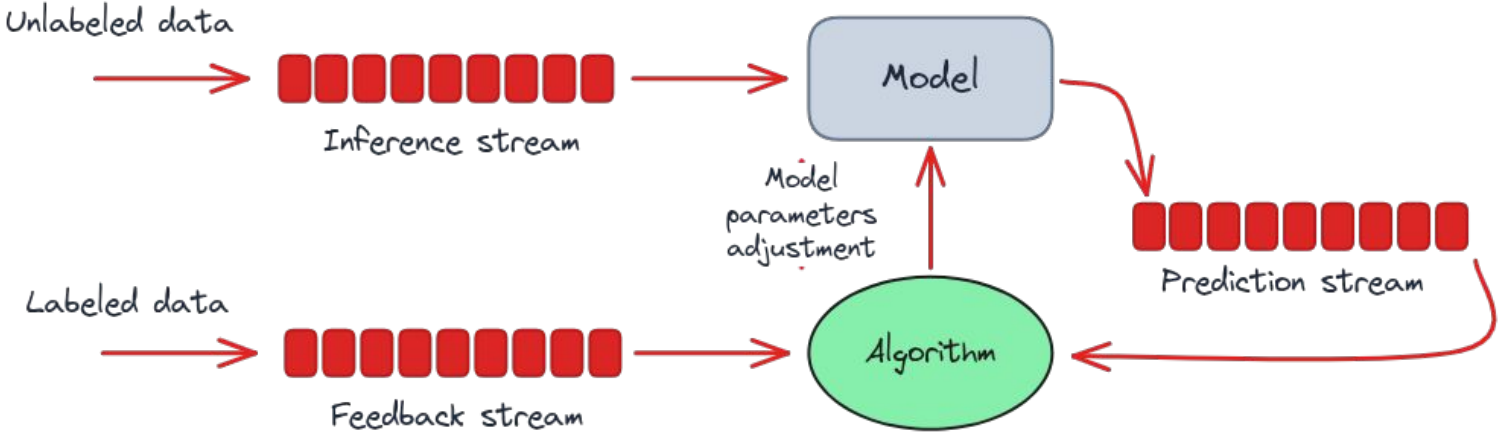


Prepare and enhance data in real-time:

- **Clean & Transform** - filtering, scaling, encoding, normalization
- **Generate & Augment** - create features, add external context
- **Complete & Label** - fill missing values, add labels for training

Tools: Kafka Streams, Apache Flink, Kafka Connect

Pattern 6: Online Learning System



Continuous model updates:

1. **Inference Stream** - unlabeled data flows to model
2. Model produces real-time predictions
3. **Feedback Stream** - true outcomes arrive as labeled data
4. **Prediction Stream** - algorithm compares predictions with true outcome
5. Model parameters adjusted on the fly

Benefits of Online Learning

Why online learning matters:

- Model adapts to data changes continuously
- No need for full retraining cycles
- Learns incrementally from each new data point
- Improves performance with every labeled outcome
- Responds quickly to distribution shifts

Perfect for dynamic, evolving environments

Part 2: LLM & RAG Patterns

Streaming for Large Language Models

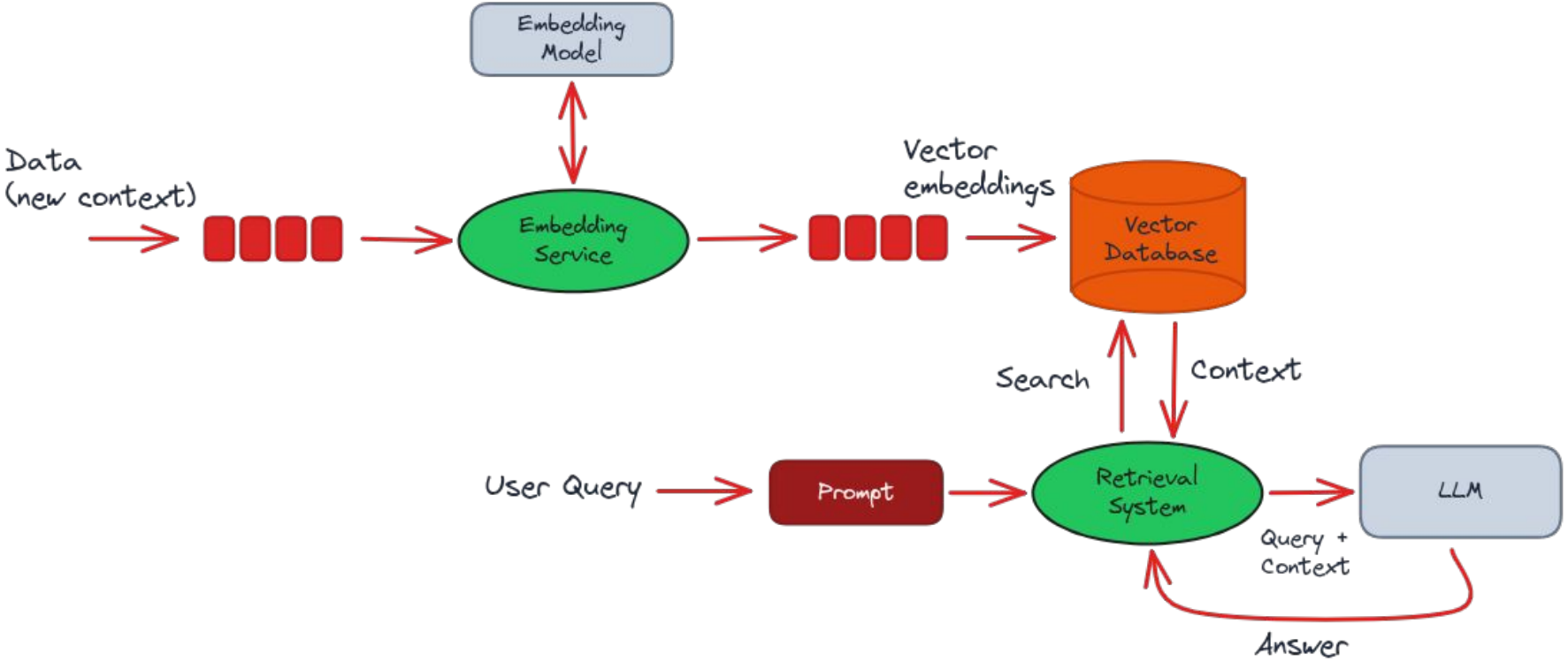
The LLM Challenge

Foundation models face limitations:

- Trained on historical internet data (quickly outdated)
- Lack current and accurate information
- Suffer from "hallucinations"
- Struggle with domain-specific knowledge
- No access to real-time business context

Solution: Retrieval Augmented Generation (RAG) with streaming data

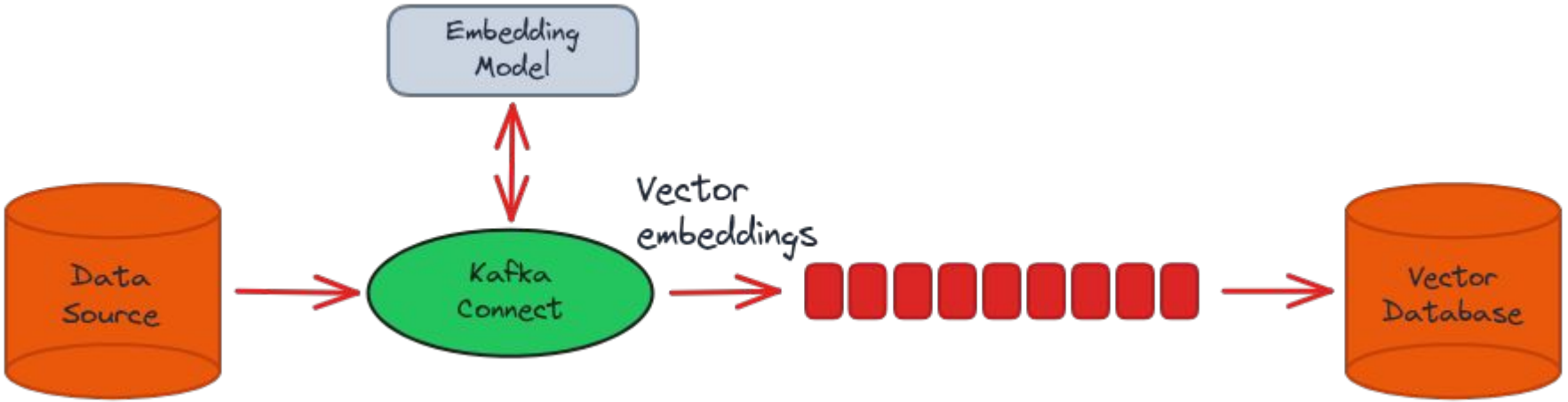
Pattern 7: RAG Architecture with Kafka



Benefits: scalability, decoupling components, real-time processing, fault tolerance

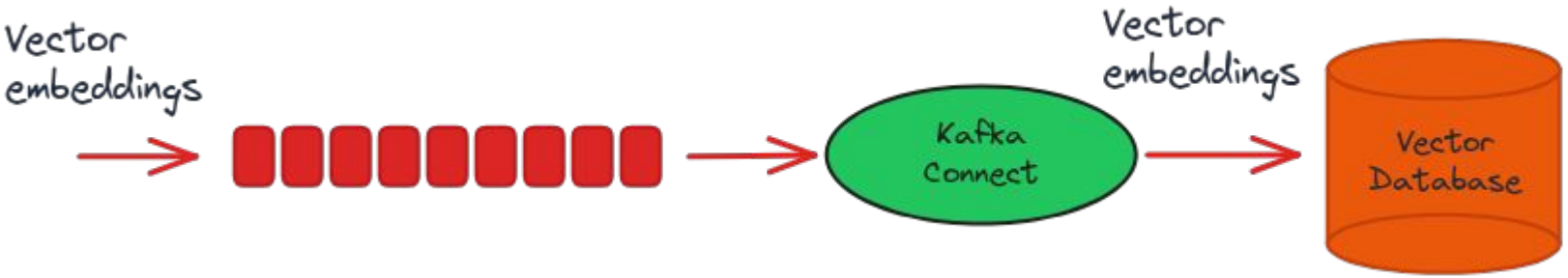
Pattern 8: Kafka Connect for RAG - Option 1

Source Connector pulls data and produces embeddings



Pattern 8: Kafka Connect for RAG - Option 2

Sink Connector reads embeddings and writes to vector database



Pattern 8: Kafka Connect - Combined Architecture

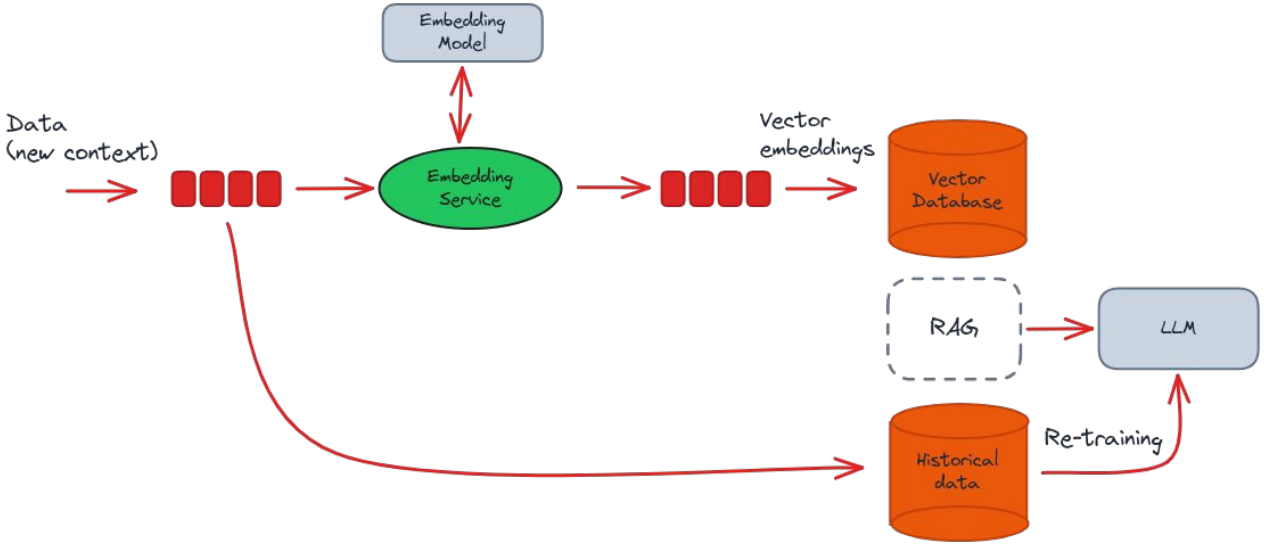
Combines both: Source pulls data → Stream processes → Sink writes to vector DB

Complete pipeline:

- Source connector pulls raw data from external source
- Writes data into Kafka topic
- Kafka Streams/Flink reads and preprocesses data
- Generates embeddings, writes to another topic
- Sink connector reads embeddings and writes to vector database

Benefits: Automation, scalability, maintainability, replay

Pattern 9: Historical Data Feed for LLM Training



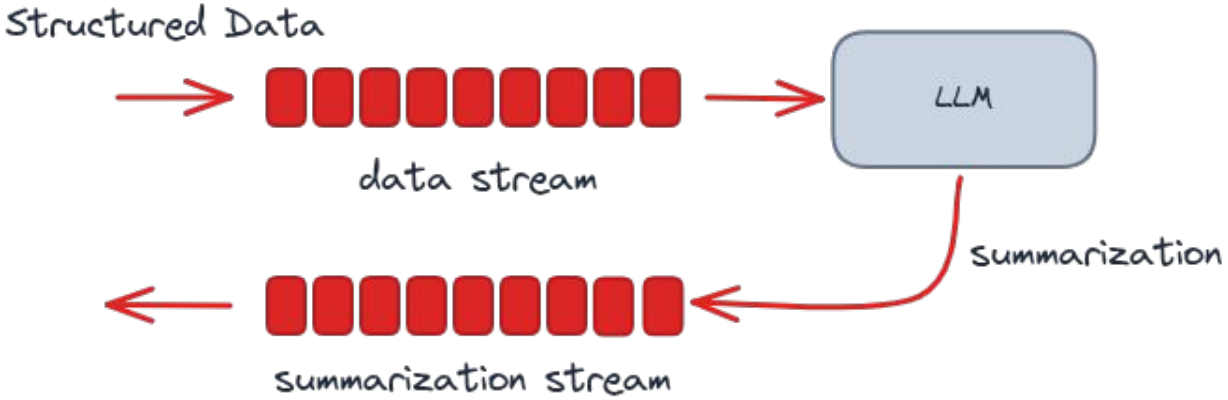
Kafka topics become training data repositories:

- Accumulate for retraining the foundational model
- Create specialized versions of LLMs
- Enable LLM use outside of RAG pattern

Part 3: LLM-Driven Data Streaming

How LLM applications leverage Kafka for data movement

Pattern 10: Summarization



Structured data → Natural language

- Kafka brings well-structured data
- Schema Registry stores data schemas
- LLM generates summaries from structured data

Transform data into human-readable content

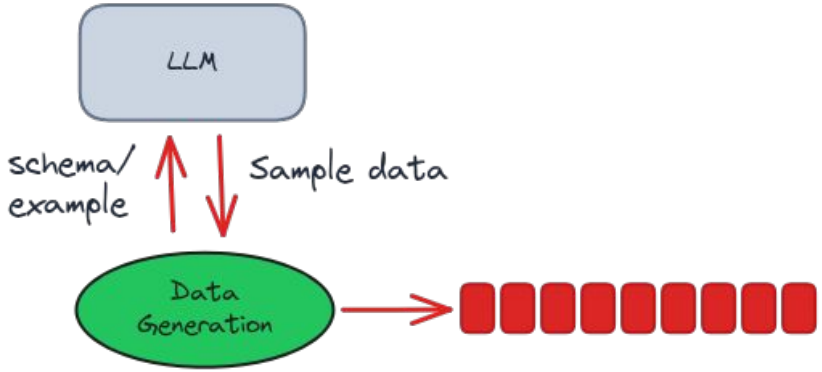
Pattern 11: Structured Data Extraction

Natural language → Structured data

- Extract customer data from emails, chats
- LLM parses unstructured text
- Write structured data to Kafka topics
- Microservices process the structured data

Reverse of summarization pattern

Pattern 12: Data Generation

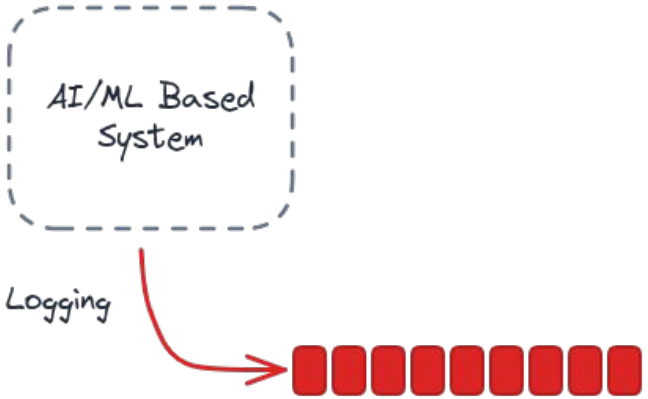


Synthetic data creation:

- Application supplies schema or example
- LLM generates sample data adhering to structure
- Synthetic structured data fed into stream
- Use for testing, development, data augmentation

AI-powered test data generation

Pattern 13: Logging & Auditing



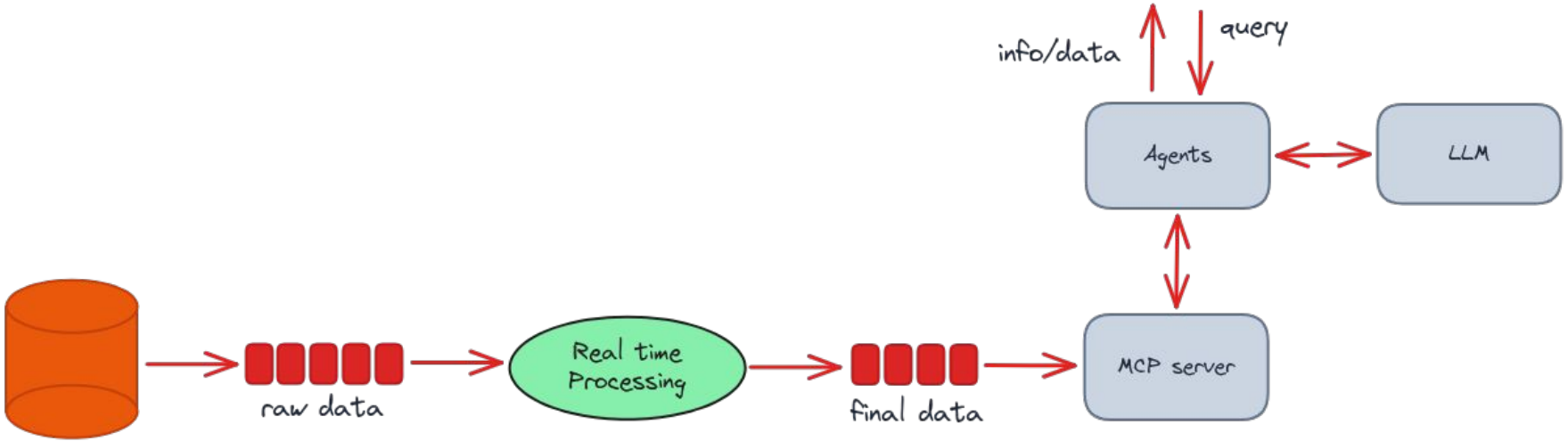
AI/ML applications are still applications

- Need logging like any other software
- Require auditing and compliance tracking
- Apache Kafka heavily used for log aggregation
- Centralized monitoring and observability

Part 4: Agentic AI

Streaming patterns for intelligent agents

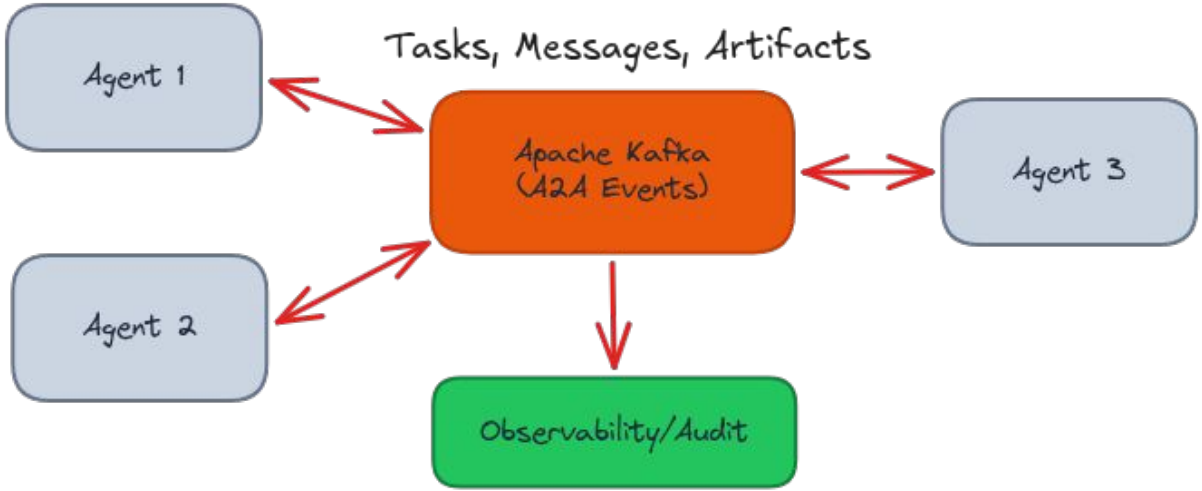
Pattern 14: Real-Time Data Feed for Agent Tools



Key concepts:

- Agents use tools via function calling (MCP)
- Kafka feeds real-time data into tool layer
- Agents combine live context with LLM reasoning

Pattern 15: Agent-to-Agent (A2A) Protocol over Kafka



Asynchronous agent communication via Kafka:

- Agents exchange **Tasks**, **Messages**, and **Artifacts** as events
- Kafka provides durable, auditable event backbone
- Multiple agents can consume and respond to same events
- Observability and audit trails built-in

A2A Architecture Benefits

Why Kafka for Agent Communication?

- **Asynchronous coordination** - agents operate independently without blocking
- **Event-driven architecture** - agents react to events in real-time
- **Multi-agent orchestration** - coordinate complex workflows across agents
- **Durability & replay** - complete audit trails, resume from last event
- **Multi-consumer support** - multiple agents can process same task streams
- **Observability** - monitor all agent activities without interference

Kafka transforms agents from isolated processes into coordinated systems

Key Takeaways

When Can We Trust Expert Predictions?

Insights from Daniel Kahneman's "Thinking, Fast and Slow" - Chapter 22

Two Essential Conditions for Valid Expert Intuition:

- **Regular Environment:** Stable, predictable patterns that can be learned and recognized
- **Immediate Feedback:** Quick feedback loops that enable genuine skill development through practice

Without both conditions: Experts develop *confidence without competence*

From Human Intuition to Machine Prediction

Streaming AI/ML Systems Meet Kahneman's Conditions:

- **Regular Environment:** Continuous data flows provide fresh, consistent patterns
- **Immediate Feedback:** Real-time validation streams enable adaptive learning
- **Result:** Genuine predictive skill that evolves with changing data

Key Insight: ML models, like human experts, need both regularity and feedback to develop valid predictions.

Key Takeaways

Apache Kafka: The Streaming Backbone for Modern AI/ML

- **Streaming enables continuous ML**
- **Real-time feedback loops matter**
- **Patterns are composable**
- **Kafka provides the foundation**

Thank You!

Questions?

Streaming AI/ML with Apache Kafka
Real-Time Patterns for Modern Intelligence

Full article: paolopatierno.dev
Twitter/X: @ppatierno
LinkedIn: linkedin.com/in/paolopatierno